



杭州电子科技大学
《编译原理课程实践》
实验报告

题 目：_____语义分析_____

学 院：_____计算机学院_____

专 业：_____计算机科学与技术_____

班 级：_____22052312_____

学 号：_____22050239_____

姓 名：_____谭文轩_____

完成日期：_____2024.12.12_____

1 实验目的

1. 在词法和语法分析的基础上，编写一个程序，对使用SysY语言书写的源代码进行静态语义检查，并打印分析结果。

2 实验内容与实验要求

1. 程序要能够查出SysY源代码中可能包含的语义错误。本次实验中，SysY语言的语义特性符合SysY语言定义(2022版)中的语义约束描述。

3 设计方案与算法描述

3.1 整体流程

根据4.3实验的现有代码，在合适位置调用报错函数，在发生错误时报错函数输出相关信息，为便于理解，错误输出以中文形式输出。

3.2 报错函数实现

```
void yyerror(const char *fmt, ...)
{
    extern int yylineno;
    extern char *yytext;
    extern int yychar;

    va_list args;
    va_start(args, fmt);

    if(fmt!="syntax error")
        switch(va_arg(args, int))
        {
            case VarUndeclared:
                fprintf(stderr, "错误代号: %d ,发生了 变量未定义 错误, 错误发生在第 %d 行, 变量名是 %s\n",VarUndeclared,yylineno,fmt);
                break;
            case VarRedeclared:
                fprintf(stderr, "错误代号: %d ,发生了 变量重定义 错误, 错误发生在第 %d 行, 变量名是 %s\n",VarRedeclared,yylineno,fmt);
                break;
            case FuncUndeclared:
                fprintf(stderr, "错误代号: %d ,发生了 函数未定义 错误, 错误发生在第 %d 行, 函数名是 %s\n",FuncUndeclared,yylineno,fmt);
                break;
            case FuncRedeclared:
                fprintf(stderr, "错误代号: %d ,发生了 函数重定义 错误, 错误发生在第 %d 行, 函数名是 %s\n",FuncRedeclared,yylineno,fmt);
                break;
            case UseVarAsFunc:
```

```

        fprintf(stderr, "错误代号: %d ,发生了 将变量用作函数 错误, 错误发生在第
%d 行, 变量名是 %s\n", UseVarAsFunc, yylineno, fmt);
        break;
    case UseFuncAsVar:
        fprintf(stderr, "错误代号: %d ,发生了 将函数用作变量 错误, 错误发生在第
%d 行, 函数名是 %s\n", UseFuncAsVar, yylineno, fmt);
        break;
    case Stmt_Error:
        fprintf(stderr, "错误代号: %d ,发生了 语义 错误, 错误发生在第 %d 行, 该
错误可由多种原因导致, 请检查代码\n", Stmt_Error, yylineno);
        break;
    default:
        fprintf(stderr, "发生了 未知 错误, 错误发生在第 %d 行, 名为
%s\n", yylineno, fmt);
    }

    va_end(args);

    error_flag = true;
}

```

3.3 报错函数调用

由于报错函数调用数量多，在此处仅举具有典型代表意义的几处，全部调用详见源代码

3.3.1 变量未定义

```

LVa1: ID ExpArray{
if(check_symbol($1, Func)) yyerror($1, UseFuncAsVar);
else if(!check_symbol($1, Var) &&!check_symbol($1, Param))yyerror($1,
VarUndeclared);
$$ = append(LVa1, NULL, NULL, $2, 0, 0, $1, NonType);
};

```

检查第一个符号（\$1）是否既不是变量（Var）也不是参数（Param）。如果都不是，那么说明这个符号未声明就被使用了，因此调用 `yyerror` 来报告未声明变量的错误。

3.3.2 变量重定义

```

ConstDef: ID ConstExpArray ASSIGN ConstInitVal{
if(!check_symbol($1, var)) add_symbol($1, var);
else yyerror($1, VarRedeclared);
$$ = append(ConstDef, NULL, $2, $4, 0, 0, $1, NonType);
}

```

这个条件检查第一个符号（\$1），它是一个标识符（ID），是否已经被声明为一个变量（Var）。如果已经被声明，那么再次声明同一个变量就是重复声明（redeclaration）

3.3.3 函数未定义

```
UnaryExp: PrimaryExp{
    $$ = append(UnaryExp, NULL, NULL, $1, PrimaryExp, 0, NULL, NonType); }
    | ID LP RP{
    if(check_symbol($1, Var)) yyerror($1, UseVarAsFunc);
    else if(!check_symbol($1, Func)) yyerror($1, FuncUndeclared);
    $$ = append(UnaryExp, NULL, NULL, NULL, FuncRParams, 0, $1, NonType);
```

这个条件检查第一个符号（\$1）是否没有被声明为一个函数（Func）。如果标识符没有被声明为函数，但是在这个上下文中被当作函数使用，这是不允许的，因此调用 `yyerror` 来报告函数未声明的错误。

3.3.4 变量重定义

```
FuncDef: INT ID LP RP Block{
    if(!check_symbol($2, Func))add_symbol($2, Func);
    else yyerror($2, FuncRedeclared);
    $$ = append(FuncDef, NULL, NULL, $5, 0, 0, $2, Int);
```

如果第二个符号（\$2）已经被声明为一个函数，那么再次声明同一个函数就是重复声明（redeclaration），这是不允许的，因此调用 `yyerror` 来报告函数重复声明的错误。

3.3.5 将变量用作函数

```
UnaryExp: PrimaryExp{
    $$ = append(UnaryExp, NULL, NULL, $1, PrimaryExp, 0, NULL, NonType); }
    | ID LP RP{
        if(check_symbol($1, Var)) yyerror($1, UseVarAsFunc);
        else if(!check_symbol($1, Func)) yyerror($1, FuncUndeclared);
    $$ = append(UnaryExp, NULL, NULL, NULL, FuncRParams, 0, $1, NonType);
```

当第一个符号（\$1）是一个标识符，并且这个标识符已经被声明为一个变量（Var），但是在这个上下文中被当作函数使用时（即后面跟着括号 LP RP），这是错误的，因为变量不能像函数那样被调用。这时会调用 `yyerror` 函数，传入错误信息 `UseVarAsFunc`，表明尝试将变量当作函数使用。

3.3.6 将函数用作变量

```
LVal: ID ExpArray{
    if(check_symbol($1, Func)) yyerror($1, UseFuncAsVar);
    else if(!check_symbol($1, Var) &&!check_symbol($1, Param))yyerror($1,
    VarUndeclared);
    $$ = append(LVal, NULL, NULL, $2, 0, 0, $1, NonType);
};
```

当第一个符号（\$1）是一个标识符，并且这个标识符已经被声明为一个函数（Func），但是在这个上下文中被当作左值（LVal）使用时，这是错误的，因为函数不能像变量那样被赋值或作为左值。这时会调用 `yyerror` 函数，传入错误信息 `UseFuncAsVar`，表明尝试将函数当作变量使用。

3.3.7 语义错误

```
if(error_cur_line != yylineno){
    yyerror("");
    error_cur_line = yylineno;
}
```

当条件为真，即检测到错误发生在不同的行时，会调用 `yyerror` 函数。这里传入的第一个参数是一个空字符串，第二个参数是 `Stmt_Error`，这可能是一个预定义的错误代码或错误信息，用于指示发生了语句错误。

3.3.8 无效的字符

```
{ID}      { yylval.str_val = strdup(yytext); return ID; }
.         { printf("错误类型: A , 发现了无效的字符: %s , 错误出现在第 %d 行\n",
yytext,yylineno); error_flag = true; }
```

3.3.9 无效的八进制/十六进制数

```
else if (yytext[0] == '0')
{
    for (int i = 1; yytext[i] != '\0'; i++) {
        if (yytext[i] < '0' || yytext[i] > '7')
        {
            fprintf(stderr, "错误类型: A , 发现了无效的八进制数: %s , 错误出现在第 %d 行\n",yytext,yylineno);
            error_flag = true;
        }
    }
    sscanf(yytext, "%o", &value);
}
else
    sscanf(yytext, "%d", &value);
yylval.int_val = value;
return INT_LIT;
}
```

```
if (yytext[0] == '0' && (yytext[1] == 'x' || yytext[1] == 'X'))
{
    for (int i = 2; yytext[i] != '\0'; i++) {
        if (yytext[i] < '0' || (yytext[i] > '9' && yytext[i] < 'A') ||
(yytext[i] > 'F' && yytext[i] < 'a') || yytext[i] > 'f')
        {
            fprintf(stderr, "错误类型: A , 发现了无效的十六进制数: %s , 错误出现在第 %d 行\n",yytext,yylineno);
            error_flag = true;
        }
    }
    sscanf(yytext, "%x", &value);
}
```

4 测试结果

3.3.1 变量未定义

测试样例	测试结果
<pre>1 /* 2 int add(int x) 3 { 4 return x + 1; 5 } 6 */ 7 8 int main() 9 { 10 int a = 0; 11 return add(a); 12 }</pre>	错误代号：3 ,发生了 函数未定义 错误，错误发生在第 11 行，函数名是 add

3.3.2 变量重定义

测试样例	测试结果
<pre>1 int main() 2 { 3 return 0; 4 } 5 6 int main() 7 { 8 return 1; 9 }</pre>	错误代号：4 ,发生了 函数重定义 错误，错误发生在第 9 行，函数名是 main

3.3.3 函数未定义

测试样例	测试结果
<pre>1 int main() 2 { 3 a = 3; 4 return a; 5 } 6 7 int add(int n) 8 { 9 return x + 1; 10 }</pre>	错误代号：1 ,发生了 变量未定义 错误，错误发生在第 3 行，变量名是 a 错误代号：1 ,发生了 变量未定义 错误，错误发生在第 4 行，变量名是 a 错误代号：1 ,发生了 变量未定义 错误，错误发生在第 9 行，变量名是 x

3.3.4 变量重定义

测试样例	测试结果
<pre>1 const int a = 0; 2 int a = 1; 3 const float a = 0.1;</pre>	错误代号：2 ,发生了 变量重定义 错误，错误发生在第 2 行，变量名是 a 错误代号：2 ,发生了 变量重定义 错误，错误发生在第 3 行，变量名是 a

3.3.5 将变量用作函数

测试样例	测试结果
<pre>1 const int add = 0; 2 3 int main() 4 { 5 int a = 0; 6 return add(a); 7 }</pre>	错误代号：5 ,发生了 将变量用作函数 错误，错误发生在第 6 行，变量名是 add

3.3.6 将函数用作变量

测试样例	测试结果
<pre>1 int function() 2 { 3 return 1; 4 } 5 6 int main() 7 { 8 int a = 0; 9 return a + function; 10 }</pre>	错误代号：6 ,发生了 将函数用作变量 错误，错误发生在第 9 行，函数名是 function

3.3.7 语义错误

测试样例	测试结果
<pre>1 int main() 2 { 3 int a[10][12]; 4 int i; 5 a[5, 3] = 5; 6 if (a[1][2] == 0) 7 i = 1 8 else 9 i = 0; 10 }</pre>	错误代号：7 ,发生了 语义 错误，错误发生在第 5 行，该错误可由多种原因导致，请检查代码 错误代号：7 ,发生了 语义 错误，错误发生在第 8 行，该错误可由多种原因导致，请检查代码

3.3.8 无效的八进制/十六进制数

测试样例	测试结果
<pre>1 int main() 2 { 3 int a = 0xg; 4 int b = 09; 5 return a + b; 6 }</pre>	错误类型: A , 发现了无效的十六进制数: 0xg , 错误出现在第 3 行 错误类型: A , 发现了无效的八进制数: 09 , 错误出现在第 4 行

5 源代码

```
%{

#include <stdio.h>
#include <string.h>
#include <stdbool.h>
#include <stdarg.h>
#include "cst.h"
#include "lex.yy.h"

int yylex(void);

typedef const enum Error_type{
    Undefined,
    VarUndeclared,
    VarRedeclared,
    FuncUndeclared,
    FuncRedeclared,
    UseVarAsFunc,
    UseFuncAsVar,
    Stmt_Error
} Error_type;

void yyerror(const char *, ...);

extern bool error_flag;

node *root;

int error_cur_line = -1;

extern Symbol *symbol_table;

}%

%union {
    int int_val;
    float float_val;
    char *str_val;
    struct Node *node_val;
}
```

```

%token <str_val> ID
%token <int_val> INT_LIT
%token <float_val> FLOAT_LIT

%token <int_val> INT FLOAT VOID CONST RETURN IF ELSE WHILE BREAK CONTINUE LP RP
LB RB LC RC COMMA SEMICN
%token <int_val> MINUS NOT ASSIGN PLUS MUL DIV MOD AND OR EQ NE LT LE GT GE

%type <node_val> CompUnit ConstDecl VarDecl FuncDef ConstDef ConstInitVal VarDef
InitVal FuncFParam ConstExpArray Block
%type <node_val> Root BlockItem Stmt LVal PrimaryExp UnaryExp FuncRParams MulExp
Exp RelExp EqExp LAndExp Cond ConstExp
%type <node_val> ExpArray AddExp LOrExp InitVals

%nonassoc ELSE

%start Root

%%

Root: CompUnit { root = append(Root, NULL, NULL, $1, 0, 0, NULL, NonType); };

CompUnit: ConstDecl { $$ = append(CompUnit, NULL, NULL, $1, 0, 0,
NULL, NonType); }
        | VarDecl { $$ = append(CompUnit, NULL, NULL, $1, 0, 0,
NULL, NonType); }
        | FuncDef { $$ = append(CompUnit, NULL, NULL, $1, 0, 0,
NULL, NonType); }
        | ConstDecl CompUnit { $$ = append(CompUnit, $2, NULL, $1, 0, 0,
NULL, NonType); }
        | VarDecl CompUnit { $$ = append(CompUnit, $2, NULL, $1, 0, 0,
NULL, NonType); }
        | FuncDef CompUnit { $$ = append(CompUnit, $2, NULL, $1, 0, 0,
NULL, NonType); }
        ;

ConstDecl: CONST INT ConstDef SEMICN { $$ = append(ConstDecl, NULL, NULL, $3,
0, 0, NULL, Int); }
        | CONST FLOAT ConstDef SEMICN { $$ = append(ConstDecl, NULL, NULL,
$3, 0, 0, NULL, Float); }
        ;

ConstDef: ID ConstExpArray ASSIGN ConstInitVal {

    if(!check_symbol($1, Var))

        add_symbol($1, Var);

        else

            yyerror($1, VarRedeclared);

            $$ =
append(ConstDef, NULL, $2, $4, 0, 0, $1, NonType);

}
    | ID ConstExpArray ASSIGN ConstInitVal COMMA ConstDef {

```

```

if(!check_symbol($1, Var))

add_symbol($1, Var);

else

yyerror($1, VarRedeclared);

$$ =

append(ConstDef, $6, $2, $4, 0, 0, $1, NonType);

}

;

ConstExpArray: /* empty */ { $$ = NULL; }
| LB ConstExp RB ConstExpArray { $$ = append(ConstExpArray, $4,
NULL, $2, 0, 0, NULL, NonType); }
;

ConstInitVal: ConstExp { $$ =
append(ConstInitVal, NULL, NULL, $1, 0, 0, NULL, NonType); }
| LC RC { $$ =
append(ConstInitVal, NULL, NULL, NULL, 0, 0, NULL, NonType); }
| LC ConstInitVal RC { $$ =
append(ConstInitVal, NULL, NULL, $2, 0, 0, NULL, NonType); }
| LC ConstInitVal COMMA ConstInitVal RC { $$ =
append(ConstInitVal, $4, NULL, $2, 0, 0, NULL, NonType); }
;

ConstExp: MulExp { $$ = append(ConstExp, NULL, NULL, $1, 0, 0,
NULL, NonType); }
| MulExp PLUS Exp { $$ = append(ConstExp, $3, NULL, $1, PLUS, 0,
NULL, NonType); }
| MulExp MINUS Exp { $$ = append(ConstExp, $3, NULL, $1, MINUS, 0,
NULL, NonType); }
;

VarDecl: INT VarDef SEMICN { $$ = append(VarDecl, NULL, NULL, $2, 0, 0,
NULL, Int); }
| FLOAT VarDef SEMICN { $$ = append(VarDecl, NULL, NULL, $2, 0, 0,
NULL, Float); }
;

VarDef: ID ConstExpArray {
if(!check_symbol($1,
Var))
add_symbol($1,
Var);
else
yyerror($1,
VarRedeclared);
$$ = append(VarDef,
NULL, $2, NULL, 0, 0, $1, NonType);
}
| ID ConstExpArray ASSIGN InitVal {
if(!check_symbol($1,
Var))

```

```

                                add_symbol($1,
Var);
                                else
                                yyerror($1,
VarRedeclared);
                                $$ = append(VarDef,
NULL, $2, $4, 0, 0, $1, NonType);
                                }
                                | ID ConstExpArray COMMA VarDef
                                {
                                if(!check_symbol($1,
Var))
                                add_symbol($1,
Var);
                                else
                                yyerror($1,
VarRedeclared);
                                $$ = append(VarDef,
$4, $2, NULL, 0, 0, $1, NonType);
                                }
                                | ID ConstExpArray ASSIGN InitVal COMMA VarDef
                                {
                                if(!check_symbol($1,
Var))
                                add_symbol($1,
Var);
                                else
                                yyerror($1,
VarRedeclared);
                                $$ = append(VarDef,
$6, $2, $4, 0, 0, $1, NonType);
                                }
                                ;

InitVal: Exp
                                { $$ = append(InitVal, NULL, NULL, $1, Exp, 0, NULL,
NonType); }
                                | LC RC
                                { $$ = append(InitVal, NULL, NULL, NULL, InitVals, 0,
NULL, NonType); }
                                | LC InitVals RC { $$ = append(InitVal, NULL, NULL, $2, InitVals, 0,
NULL, NonType); }
                                ;

InitVals: InitVal
                                { $$ = append(InitVals, NULL, NULL, $1,
0, 0, NULL, NonType); }
                                | InitVal COMMA InitVals
                                { $$ = append(InitVals, $3, NULL, $1, 0,
0, NULL, NonType); }
                                ;

FuncDef: INT ID LP RP Block
                                {
                                if(!check_symbol($2, Func))
                                add_symbol($2, Func);
                                else
                                yyerror($2,
FuncRedeclared);
                                $$ = append(FuncDef, NULL,
NULL, $5, 0, 0, $2, Int);
                                }

```

```

| FLOAT ID LP RP Block
{
    if(!check_symbol($2, Func))
        add_symbol($2, Func);
    else
        yyerror($2,

FuncRedeclared);

    $$ = append(FuncDef, NULL,

NULL, $5, 0, 0, $2, Float);
}

| VOID ID LP RP Block
{
    if(!check_symbol($2, Func))
        add_symbol($2, Func);
    else
        yyerror($2,

FuncRedeclared);

    $$ = append(FuncDef, NULL,

NULL, $5, 0, 0, $2, Void);
}

| INT ID LP FuncFParam RP Block
{
    if(!check_symbol($2, Func))
        add_symbol($2, Func);
    else
        yyerror($2,

FuncRedeclared);

    $$ = append(FuncDef, NULL,

$4, $6, 0, 0, $2, Int);
}

| FLOAT ID LP FuncFParam RP Block
{
    if(!check_symbol($2, Func))
        add_symbol($2, Func);
    else
        yyerror($2,

FuncRedeclared);

    $$ = append(FuncDef, NULL,

$4, $6, 0, 0, $2, Float);
}

| VOID ID LP FuncFParam RP Block
{
    if(!check_symbol($2, Func))
        add_symbol($2, Func);
    else
        yyerror($2,

FuncRedeclared);

    $$ = append(FuncDef, NULL,

$4, $6, 0, 0, $2, Void);
}

;

FuncFParam: INT ID
{
    if(!check_symbol($2, Param))

    add_symbol($2, Param);

    $$ =

append(FuncFParam, NULL, NULL, NULL, 0, 0, $2, Int);
}

```

```

| FLOAT ID {

if(!check_symbol($2,Param))

add_symbol($2,Param);

append(FuncFParam, NULL, NULL, NULL, 0, 0, $2, Float);

}

| INT ID LB RB ExpArray {

if(!check_symbol($2,Param))

add_symbol($2,Param);

append(FuncFParam, NULL, NULL, $5, 0, 0, $2, Int);

}

| FLOAT ID LB RB ExpArray {

if(!check_symbol($2,Param))

add_symbol($2,Param);

append(FuncFParam, NULL, NULL, $5, 0, 0, $2, Float);

}

| INT ID COMMA FuncFParam {

if(!check_symbol($2,Param))

add_symbol($2,Param);

append(FuncFParam, $4, NULL, NULL, 0, 0, $2, Int);

}

| FLOAT ID COMMA FuncFParam {

if(!check_symbol($2,Param))

add_symbol($2,Param);

append(FuncFParam, $4, NULL, NULL, 0, 0, $2, Float);

}

| INT ID LB RB ExpArray COMMA FuncFParam {

if(!check_symbol($2,Param))

add_symbol($2,Param);

append(FuncFParam, $7, NULL, $5, 0, 0, $2, Int);

}

| FLOAT ID LB RB ExpArray COMMA FuncFParam {

if(!check_symbol($2,Param))

add_symbol($2,Param);

append(FuncFParam, $7, NULL, $5, 0, 0, $2, Float);

```

```

    }

;

Block: LC BlockItem RC { $$ = append(Block, NULL, NULL, $2, 0, 0, NULL,
NonType); };

BlockItem: /* empty */      { $$ = NULL; }
        | ConstDecl BlockItem { $$ = append(BlockItem, $2, NULL, $1, 0, 0,
NULL, NonType); }
        | VarDecl BlockItem  { $$ = append(BlockItem, $2, NULL, $1, 0, 0,
NULL, NonType); }
        | Stmt BlockItem     { $$ = append(BlockItem, $2, NULL, $1, 0, 0,
NULL, NonType); }
        ;

Stmt: LVal ASSIGN Exp SEMICN { $$ = append(AssignStmt, $3, NULL, $1, 0,
0, NULL, NonType); }
    | Exp SEMICN              { $$ = append(ExpStmt, NULL, NULL, $1, 0,
0, NULL, NonType); }
    | Block                   { $$ = append(BlockStmt, NULL, NULL, $1, 0,
0, NULL, NonType); }
    | IF LP Cond RP Stmt      { $$ = append(IfStmt, $5, NULL, $3, 0, 0,
NULL, NonType); }
    | IF LP Cond RP Stmt ELSE Stmt { $$ = append(IfElseStmt, $7, $5, $3, 0, 0,
NULL, NonType); }
    | WHILE LP Cond RP Stmt   { $$ = append(WhileStmt, $5, NULL, $3, 0,
0, NULL, NonType); }
    | BREAK SEMICN            { $$ = append(BreakStmt, NULL, NULL, NULL,
0, 0, NULL, NonType); }
    | CONTINUE SEMICN         { $$ = append(ContinueStmt, NULL, NULL,
NULL, 0, 0, NULL, NonType); }
    | RETURN Exp SEMICN       { $$ = append(ReturnStmt, NULL, NULL, $2,
0, 0, NULL, NonType); }
    | RETURN SEMICN           { $$ = append(BlankReturnStmt, NULL, NULL,
NULL, 0, 0, NULL, NonType); }
    | error                    {
                                if(error_cur_line != yylineno)
                                {
                                    yyerror("", Stmt_Error);
                                    error_cur_line = yylineno;
                                }
                                yyclearin;
                                yyerrok;
                            }
    ;

Exp: AddExp { $$ = append(Exp, NULL, NULL, $1, 0, 0, NULL, NonType); };

AddExp: MulExp { $$ = append(AddExp, NULL, NULL, $1, Mul, 0,
NULL, NonType); }
    | MulExp PLUS AddExp { $$ = append(AddExp, $3, NULL, $1, Plus, 0,
NULL, NonType); }
    | MulExp MINUS AddExp { $$ = append(AddExp, $3, NULL, $1, Minus, 0,
NULL, NonType); }
    ;

```

```
MulExp: UnaryExp { $$ = append(MulExp, NULL, NULL, $1, UnaryExp,
0, NULL, NonType); }
    | UnaryExp MUL MulExp { $$ = append(MulExp, $3, NULL, $1, Mul, 0,
NULL, NonType); }
    | UnaryExp DIV MulExp { $$ = append(MulExp, $3, NULL, $1, Div, 0,
NULL, NonType); }
    | UnaryExp MOD MulExp { $$ = append(MulExp, $3, NULL, $1, Mod, 0,
NULL, NonType); }
;

UnaryExp: PrimaryExp { $$ = append(UnaryExp, NULL, NULL, $1,
PrimaryExp, 0, NULL, NonType); }
    | ID LP RP {
        if(check_symbol($1, var))
            yyerror($1, UseVarAsFunc);
        else if(!check_symbol($1, Func))
            yyerror($1, FuncUndeclared);
        $$ = append(UnaryExp, NULL, NULL, NULL,
FuncRParams, 0, $1, NonType);
    }
    | ID LP FuncRParams RP {
        if(check_symbol($1, var))
            yyerror($1, UseVarAsFunc);
        else if(!check_symbol($1, Func))
            yyerror($1, FuncUndeclared);
        $$ = append(UnaryExp, NULL, NULL, $3,
FuncRParams, 0, $1, NonType);
    }
    | PLUS UnaryExp { $$ = append(UnaryExp, NULL, NULL, $2, PLUS, 0,
NULL, NonType); }
    | MINUS UnaryExp { $$ = append(UnaryExp, NULL, NULL, $2, MINUS,
0, NULL, NonType); }
    | NOT UnaryExp { $$ = append(UnaryExp, NULL, NULL, $2, Not, 0,
NULL, NonType); }
;

FuncRParams: Exp { $$ = append(FuncRParams, NULL, NULL,
$1, 0, 0, NULL, NonType); }
    | Exp COMMA FuncRParams { $$ = append(FuncRParams, $3, NULL, $1,
0, 0, NULL, NonType); }
;

PrimaryExp: LP Exp RP { $$ = append(PrimaryExp, NULL, NULL, $2, Exp, 0, NULL,
NonType); }
    | LVal { $$ = append(PrimaryExp, NULL, NULL, $1, LVal, 0, NULL,
NonType); }
    | INT_LIT { $$ = append(PrimaryExp, NULL, NULL, NULL, $1, 0, NULL,
Int); }
    | FLOAT_LIT { $$ = append(PrimaryExp, NULL, NULL, NULL, 0, $1, NULL,
Float); }
;

LVal: ID ExpArray {
    if(check_symbol($1, Func))
```

```

        yyerror($1, UseFuncAsVar);
    else if(!check_symbol($1,Var) &&
!check_symbol($1,Param))
        yyerror($1, varUndeclared);
    $$ = append(LVal, NULL, NULL, $2, 0, 0, $1,
NonType);
};

Cond: LOExp      { $$ = append(Cond, NULL, NULL, $1, 0, 0, NULL,
NonType); };

LOExp: LAndExp      { $$ = append(Cond, NULL, NULL, $1, 0, 0, NULL,
NonType); }
      | LAndExp OR LOExp      { $$ = append(Cond, $3, NULL, $1, OR, 0, 0,
NonType); }
      ;

LAndExp: EqExp      { $$ = append(LAndExp, NULL, NULL, $1, 0, 0,
NULL, NonType); }
      | EqExp AND LAndExp      { $$ = append(LAndExp, $3, NULL, $1, AND, 0,
NULL, NonType); }
      ;

EqExp: RelExp      { $$ = append(EqExp, NULL, NULL, $1, 0, 0, NULL,
NonType); }
      | RelExp EQ EqExp      { $$ = append(EqExp, $3, NULL, $1, EQ, 0, NULL,
NonType); }
      | RelExp NE EqExp      { $$ = append(EqExp, $3, NULL, $1, NE, 0, NULL,
NonType); }
      ;

RelExp: AddExp      { $$ = append(RelExp, NULL, NULL, $1, 0, 0, NULL,
NonType); }
      | AddExp LT RelExp      { $$ = append(RelExp, $3, NULL, $1, LT, 0, NULL,
NonType); }
      | AddExp GT RelExp      { $$ = append(RelExp, $3, NULL, $1, GT, 0, NULL,
NonType); }
      | AddExp LE RelExp      { $$ = append(RelExp, $3, NULL, $1, LE, 0, NULL,
NonType); }
      | AddExp GE RelExp      { $$ = append(RelExp, $3, NULL, $1, GE, 0, NULL,
NonType); }
      ;

ExpArray: /* empty */      { $$ = NULL; }
      | LB Exp RB ExpArray      { $$ = append(ExpArray, $4, NULL, $2, 0, 0,
NULL, NonType); }
      ;

%%

void yyerror(const char *fmt, ...)
{
    extern int yylineno;
    extern char *yytext;
    extern int yychar;

```

```

va_list args;
va_start(args, fmt);

if(fmt!="syntax error")
    switch(va_arg(args, int))
    {
        case VarUndeclared:
            fprintf(stderr, "错误代号: %d ,发生了 变量未定义 错误, 错误发生在第 %d
行, 变量名是 %s\n",VarUndeclared,yylineno,fmt);
            break;
        case VarRedeclared:
            fprintf(stderr, "错误代号: %d ,发生了 变量重定义 错误, 错误发生在第 %d
行, 变量名是 %s\n",VarRedeclared,yylineno,fmt);
            break;
        case FuncUndeclared:
            fprintf(stderr, "错误代号: %d ,发生了 函数未定义 错误, 错误发生在第 %d
行, 函数名是 %s\n",FuncUndeclared,yylineno,fmt);
            break;
        case FuncRedeclared:
            fprintf(stderr, "错误代号: %d ,发生了 函数重定义 错误, 错误发生在第 %d
行, 函数名是 %s\n",FuncRedeclared,yylineno,fmt);
            break;
        case UseVarAsFunc:
            fprintf(stderr, "错误代号: %d ,发生了 将变量用作函数 错误, 错误发生在第
%d 行, 变量名是 %s\n",UseVarAsFunc,yylineno,fmt);
            break;
        case UseFuncAsVar:
            fprintf(stderr, "错误代号: %d ,发生了 将函数用作变量 错误, 错误发生在第
%d 行, 函数名是 %s\n",UseFuncAsVar,yylineno,fmt);
            break;
        case Stmt_Error:
            fprintf(stderr, "错误代号: %d ,发生了 语义 错误, 错误发生在第 %d 行, 该
错误可由多种原因导致, 请检查代码\n",Stmt_Error,yylineno);
            break;
        default:
            fprintf(stderr, "发生了 未知 错误, 错误发生在第 %d 行, 名为
%s\n",yylineno,fmt);
    }

va_end(args);

error_flag = true;
}

int main(int argc, char **argv)
{
    if (argc < 2)
    {
        fprintf(stderr, "Usage: %s <filename>\n", argv[0]);
        return 1;
    }

    FILE *f = fopen(argv[1], "r");
    if (!f)

```

```

    {
        perror("fopen");
        return 1;
    }

    YY_BUFFER_STATE bp = yy_create_buffer(f, YY_BUF_SIZE);
    yy_switch_to_buffer(bp);

    yyparse();

    yy_delete_buffer(bp);

    fclose(f);

    if(!error_flag)
        print_tree(root,0);
    return 0;
}

```

```

%{
    #include <stdio.h>
    #include <stdbool.h>
    #include "y.tab.h"

    bool error_flag = false;
%}

%option yylineno
%option header-file="lex.yy.h"

INT          int
FLOAT        float
VOID         void
CONST        const
RETURN       return
IF           if
ELSE         else
WHILE        while
BREAK        break
CONTINUE     continue
PLUS         \+
MINUS        -
INT_LIT      [1-9]+[0-9]*|0|0[xx][0-9a-zA-Z]+|0[0-9]+
FLOAT_LIT    [0-9]+\.[0-9]*
ID           [a-zA-Z][a-zA-Z_0-9]*
SPACE        [ \t\r]*
EOL          \n
SEMICN       ;
COMMA        ,
ASSIGN       =
EQ           ==
NE           !=
LT           <
LE           <=
GT           >

```

GE	>=
STAR	*
DIV	\/
AND	&&
OR	\ \
NOT	!
LP	\(
RP	\)
LB	\[
RB	\]
LC	\{
RC	\}

%%

{SPACE}	{ /* 忽略空白字符 */ }
{EOL}	{ /* 忽略换行符 */ }

"/".* { }

"/"([^*]|(*)*([^*/])*(*))**"/" { }

{INT}	{ return INT; }
{FLOAT}	{ return FLOAT; }
{VOID}	{ return VOID; }
{SEMICN}	{ return SEMICN; }
{COMMA}	{ return COMMA; }
{ASSIGN}	{ return ASSIGN; }
{EQ}	{ return EQ; }
{NE}	{ return NE; }
{LT}	{ return LT; }
{LE}	{ return LE; }
{GT}	{ return GT; }
{GE}	{ return GE; }
{STAR}	{ return MUL; }
{DIV}	{ return DIV; }
{AND}	{ return AND; }
{OR}	{ return OR; }
{NOT}	{ return NOT; }
{LP}	{ return LP; }
{RP}	{ return RP; }
{LB}	{ return LB; }
{RB}	{ return RB; }
{LC}	{ return LC; }
{RC}	{ return RC; }
{RETURN}	{ return RETURN; }
{BREAK}	{ return BREAK; }
{CONTINUE}	{ return CONTINUE; }
{IF}	{ return IF; }
{ELSE}	{ return ELSE; }
{WHILE}	{ return WHILE; }
{PLUS}	{ return PLUS; }
{MINUS}	{ return MINUS; }
{CONST}	{ return CONST; }

```

{INT_LIT} {
    int value;
    if (yytext[0] == '0' && (yytext[1] == 'x' || yytext[1] == 'X'))
    {
        for (int i = 2; yytext[i] != '\0'; i++) {
            if (yytext[i] < '0' || (yytext[i] > '9' && yytext[i] < 'A') ||
(yytext[i] > 'F' && yytext[i] < 'a') || yytext[i] > 'f')
            {
                fprintf(stderr, "错误类型: A , 发现了无效的十六进制数: %s , 错误出
现在第 %d 行\n",yytext,yylineno);
                error_flag = true;
            }
        }
        sscanf(yytext, "%x", &value);
    }
    else if (yytext[0] == '0')
    {
        for (int i = 1; yytext[i] != '\0'; i++) {
            if (yytext[i] < '0' || yytext[i] > '7')
            {
                fprintf(stderr, "错误类型: A , 发现了无效的八进制数: %s , 错误出现
在第 %d 行\n",yytext,yylineno);
                error_flag = true;
            }
        }
        sscanf(yytext, "%o", &value);
    }
    else
        sscanf(yytext, "%d", &value);
    yylval.int_val = value;
    return INT_LIT;
}

{FLOAT_LIT} { yylval.float_val = atof(yytext); return FLOAT_LIT; }
{ID}        { yylval.str_val = strdup(yytext); return ID; }
.           { printf("错误类型: A , 发现了无效的字符: %s , 错误出现在第 %d 行\n"
,yytext,yylineno); error_flag = true; }

%%

int yywrap(void) {
    return 1;
}

```

